

Université Sorbonne Paris Nord
Ecole d'Ingénieurs Sup Galilée

COMPTE RENDU

TP5: Transactions et contrôle de concurrence

Réalisé par:

ELGARANI Hasna

2024/2025

→ Préparation de l'environnement

Avant chaque connexion, il est important de désactiver l'autocommit pour gérer explicitement les transactions. Cela se fait avec :

```
SET AUTOCOMMIT OFF;
```

Exercice 1 : Atomicité d'une transaction

1. Création de la table transaction (session S1)

```
CREATE TABLE transaction (  
  idTransaction VARCHAR2(44),  
  valTransaction NUMBER(10)  
);
```

2. Manipulations dans la session S2

Dans une deuxième session, j'ai inséré plusieurs lignes, modifié une valeur, puis supprimé une ligne dans la table transaction. Avant de valider quoi que ce soit, j'ai annulé toutes ces opérations avec :

```
ROLLBACK;
```

→ Les étapes :

- Insertion,

```
INSERT INTO transaction (idTransaction, valTransaction) VALUES ('T1', 120);  
INSERT INTO transaction (idTransaction, valTransaction) VALUES ('T2', 300);
```
- Modifié une ligne existante,

```
UPDATE transaction SET valTransaction = 150 WHERE idTransaction = 'T2';
```
- Supprimé une autre ligne,

```
DELETE FROM transaction WHERE idTransaction = 'T1';
```
- Puis annulé toutes ces opérations avec la commande ROLLBACK.

Après ce rollback, la table est revenue à son état initial, sans aucune des modifications effectuées. Cela illustre l'atomicité d'une transaction : soit toutes les opérations sont validées, soit aucune ne l'est.

3. Insertion et fermeture de session

Après avoir inséré de nouvelles lignes dans la table transaction en session S2, j'ai fermé la session avec quit; sans faire de commit. En consultant la table depuis la session S1, aucune des nouvelles données n'est visible. Cela montre que tant qu'une transaction n'est pas validée, ses modifications ne sont pas enregistrées de façon permanente.

Donc après avoir effectué un ROLLBACK, nous avons réinséré de nouvelles lignes dans la session S2.

```
INSERT INTO transaction (idTransaction, valTransaction) VALUES ('T1', 120);  
INSERT INTO transaction (idTransaction, valTransaction) VALUES ('T2', 300);
```

Cependant, sans exécuter COMMIT, nous avons fermé la session avec la commande quit;.

En consultant la table transaction depuis la session S1, nous avons constaté que **les nouvelles lignes insérées dans S2 n'étaient pas visibles**.

4. Fermeture brutale sans commit

En session S1, j'ai inséré des lignes puis fermé brutalement sqlplus, sans valider. Après reconnexion, les données saisies n'étaient pas présentes dans la table. Cela confirme que seules les transactions validées (commit) sont conservées.

5. Modification de la structure et rollback

Dans une nouvelle session, j'ai ajouté des lignes et modifié la structure de la table en ajoutant une colonne :

```
ALTER TABLE transaction ADD (val2transaction NUMBER(10));
```

Après exécution d'un ROLLBACK, la structure de la table n'a pas été annulée, seule l'insertion de données l'a été. Les commandes DDL (modification de structure) ne sont pas concernées par le rollback.

6. Conclusion – Définitions

- Session : Connexion d'un utilisateur à la base de données, pouvant contenir plusieurs transactions successives.
- Transaction : Ensemble d'opérations SQL exécutées comme une unité atomique. Elle est validée par COMMIT ou annulée par ROLLBACK.
- Valider une transaction : Rendre définitives les modifications avec COMMIT.
- Annuler une transaction : Revenir à l'état initial avec ROLLBACK.

Exercice 2 : Transactions concurrentes

Création du schéma simplifié

```
CREATE TABLE vol (  
  idVol VARCHAR2(44),  
  capaciteVol NUMBER(10),  
  nbrPlacesReserveesVol NUMBER(10)  
);
```

```
CREATE TABLE client (
  idClient VARCHAR2(44),
  prenomClient VARCHAR2(11),
  nbrPlacesReserveesClient NUMBER(10)
);
```

Isolation des transactions

Dans deux sessions différentes, j'ai inséré un vol et deux clients. J'ai ensuite effectué une réservation pour un client (T1) sans valider.

- Observation : T1 voit ses propres modifications, mais T2 ne les voit pas. Cela montre l'isolation des transactions dans Oracle (niveau READ COMMITTED par défaut).

COMMIT et ROLLBACK

Après un ROLLBACK dans T1, la base revient à l'état initial. Un commit ou rollback supplémentaire n'a plus d'effet, car chaque transaction démarre à l'état courant.

Après un COMMIT, les modifications deviennent visibles pour T2 et sont définitives (durabilité).

Validation des mises à jour avec COMMIT :

```
START TRANSACTION;
UPDATE client SET nbrPlacesReservesClient = nbrPlacesReservesClient + 1 WHERE idClient = 'C1';
UPDATE vol SET nbrPlacesReservesVol = nbrPlacesReservesVol + 1 WHERE idVol = 'V1';
COMMIT;
```

Problème d'isolation incomplète

En mode READ COMMITTED, j'ai simulé deux transactions concurrentes :

- T1 réserve 2 billets pour c1.
 - T2 réserve 3 billets pour c2.
- Après validation, chaque client a bien réservé ses billets, mais le total des places réservées dans vol est incorrect (seulement 3).
- Conclusion : L'isolation incomplète peut provoquer des incohérences (mises à jour perdues).

```
START TRANSACTION;
UPDATE client SET nbrPlacesReservesClient = nbrPlacesReservesClient + 1 WHERE idClient = 'C1';
UPDATE vol SET nbrPlacesReservesVol = nbrPlacesReservesVol + 1 WHERE idVol = 'V1';
COMMIT;
```

```

START TRANSACTION;
SELECT * FROM vol WHERE idVol = 'V1' FOR UPDATE;
SELECT * FROM client WHERE idClient = 'C2' FOR UPDATE;
UPDATE vol SET nbrPlacesReservesVol = nbrPlacesReservesVol + 3 WHERE idVol = 'V1';
UPDATE client SET nbrPlacesReservesClient = nbrPlacesReservesClient + 3 WHERE idClient = 'C2';
COMMIT;

```

Isolation complète (SERIALIZABLE)

En passant les sessions en mode **SERIALIZABLE**, j'ai refait le test précédent. Cette fois, une des transactions a été bloquée ou annulée, empêchant toute incohérence.

Cela garantit la cohérence des données, mais peut entraîner des blocages ou des rejets de transactions concurrentes.

Reprenons maintenant l'exemple d'exécution concurrente vu en cours :

Session T1 :

```

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
START TRANSACTION;
SELECT * FROM vol WHERE idVol = 'V1' FOR UPDATE;
SELECT * FROM client WHERE idClient = 'C1' FOR UPDATE;
UPDATE vol SET nbrPlacesReservesVol = nbrPlacesReservesVol + 2 WHERE idVol = 'V1';
UPDATE client SET nbrPlacesReservesClient = nbrPlacesReservesClient + 2 WHERE idClient = 'C1';
COMMIT;

```

Session T2 :

```

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
START TRANSACTION;
SELECT * FROM vol WHERE idVol = 'V1' FOR UPDATE;
SELECT * FROM client WHERE idClient = 'C2' FOR UPDATE;
UPDATE vol SET nbrPlacesReservesVol = nbrPlacesReservesVol + 3 WHERE idVol = 'V1';
UPDATE client SET nbrPlacesReservesClient = nbrPlacesReservesClient + 3 WHERE idClient = 'C2';
COMMIT;

```

Exemple d'exécution concurrente

J'ai reproduit la séquence suivante :

$r_1(d)$ $w_2(d)$ $w_2(d')$ C_2 $w_1(d')$ C_1

Cela correspond à une gestion de verrouillage à deux phases, similaire à l'algorithme vu en cours.

Conclusion

- Oracle implémente un mécanisme de verrouillage à deux phases pour garantir la cohérence des transactions concurrentes.

- Le niveau d'isolation READ COMMITTED protège contre la lecture de données non validées, mais n'empêche pas toutes les anomalies.
- Le mode SERIALIZABLE offre une isolation maximale, au prix d'une concurrence réduite.
- La gestion explicite des transactions (COMMIT, ROLLBACK) est essentielle pour garantir l'intégrité et la cohérence des données.